The `circover` Package

NHOP · Seed Selection · Circular Oversampling · Degradation Bench

Parsa Hajiannejad

Università degli Studi di Milano

2026

What is circover?

A Python library for imbalanced classification

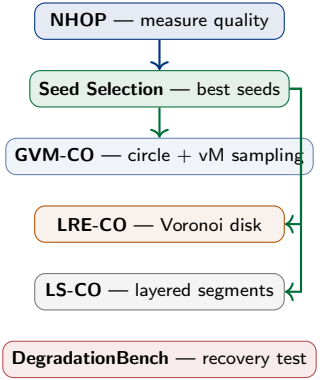
From the thesis:

"From Distributional Similarity to Causal Imbalance: NHOP, Circular Oversampling, and a Controlled Degradation Study"

Parsa Hajiannejad, Università degli Studi di Milano, 2025.

Module	Purpose
`NHOP`	Distribution similarity metric
`GeometricSeedSelector`	Geometry-preserving seed selection
`GVMCO`	Gravity-biased Von Mises oversampling
`LRECO`	Voronoi-constrained disk oversampling
`LSCO`	Layered segmental oversampling
`DegradationBench`	Controlled degradation benchmark

`pip install circover` — `import circover as cc`



Install & Quick Start

Install

```
pip install circover
```

Measure distribution fidelity (NHOP)

```
import circover as cc
import numpy as np

nhop = cc.NHOP(n_bins=30)

# Overall score: 1.0 = perfect, 0.0 = disjoint
score = nhop.score(X_original, X_synthetic)

# Per-feature breakdown
per_feat = nhop.score_per_feature(X_orig, X_synth)
tv_dist  = nhop.tv_per_feature(X_orig, X_synth)
# TV = 1 - NHOP  (exact identity)
```

Oversample (drop-in for SMOTE)

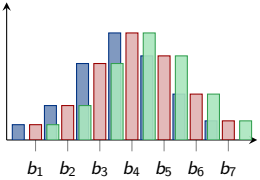
```
from imblearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

pipe = Pipeline([
    ("over", cc.GVMCO(random_state=42)),
    # swap for: LRECO(), LSCO()
    ("clf",  RandomForestClassifier()),
])
scores = cross_val_score(
    pipe, X, y, scoring="f1", cv=5)
```

Seed selection

```
sel = cc.GeometricSeedSelector(
          n_seeds=20, random_state=42)
seed_idx, best_score = sel.select(X_minority)
```

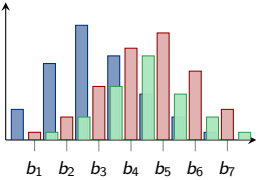
NHOP Intuition: Shared Histogram Bins (1-D)

$\mathrm{NHOP} = 1.0$ — **Identical**



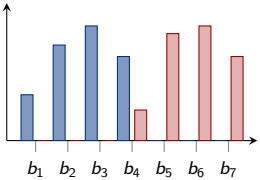
$\sum \min(p_b, q_b) = 1.00$

$\mathrm{NHOP} \approx 0.57$ — **Partial**



$\sum \min(p_b, q_b) = 0.64$ (scaled ≈ 0.57)

$\mathrm{NHOP} = 0.0$ — **Disjoint**



$\sum \min(p_b, q_b) = 0.00$

P (original) Q (synthetic) $\min(p_b, q_b)$ — shared mass

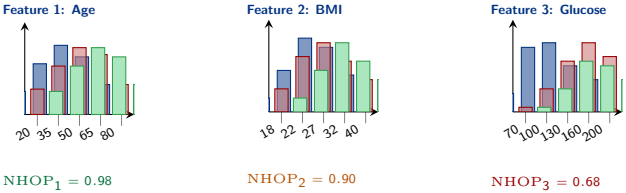
$$\mathrm{NHOP} = \sum_{b=1}^{B} \min(p_b,\ q_b)$$

where bins are computed on the *joint* range of P and Q

NHOP Intuition: Multiple Dimensions

Each feature gets its own histogram overlap score

Example: synthetic patient data ($k = 3$ features). NHOP computed **independently** on each feature's histogram.



Global NHOP = average over features:

$$\mathrm{NHOP} = \frac{1}{3}(0.98 + 0.90 + 0.68) = \mathbf{0.853}$$

What this tells you

Feature	Diagnosis
Age	0.98 — near-perfect
BMI	0.90 — small drift
Glucose	0.68 — **poor!**

Actionable: the oversampler distorted the Glucose distribution. You can now *target* that feature for tuning.

Per-feature in code:

```
nhop = cc.NHOP(n_bins=30)
pf = nhop.score_per_feature(
    X_orig, X_synth)
# array([0.98, 0.90, 0.68])
```

NHOP: When and How to Use It

1 — Did my oversampler preserve distributions?

```
import circover as cc
nhop = cc.NHOP(n_bins=30)
X_res, _ = cc.GVMCO().fit_resample(X, y)
X_synth = X_res[len(X):]          # synthetic part
print(nhop.score(X[y==1], X_synth))
# 0.91  -- GVM-CO preserves distribution well
```

2 — Which oversampler has better fidelity?

```
from imblearn.over_sampling import SMOTE
smote = SMOTE().fit_resample(X, y)
X_sm  = smote[0][len(X):]
X_gvm = cc.GVMCO().fit_resample(X,y)[0][len(X):]
print(nhop.score(X[y==1], X_sm))    # 0.77
print(nhop.score(X[y==1], X_gvm))   # 0.91 better
```

3 — Which features drift most?

```
pf = nhop.score_per_feature(X[y==1], X_gvm)
# array([0.98, 0.90, 0.68])  <- feature 3 bad!
import matplotlib.pyplot as plt
plt.bar(range(len(pf)), pf)
plt.axhline(0.9, color='red', ls='--')
plt.show()
```

4 — Use NHOP as a CV scorer

```
from sklearn.model_selection import cross_val_score

def nhop_scorer(est, X, y):
    X_r, _ = est['over'].fit_resample(X, y)
    return cc.NHOP().score(X[y==1], X_r[len(X):])

cv_nhop = cross_val_score(
    pipe, X, y, scoring=nhop_scorer, cv=5)
print(cv_nhop.mean())   # fidelity across folds
```

5 — TV distance per feature

```
# TV = 1 - NHOP  (exact identity, Theorem 4.5)
tv = nhop.tv_per_feature(X[y==1], X_gvm)
# array([0.02, 0.10, 0.32])
# Feature 3 has 32% TV distance -> investigate
```

Rule of thumb

NHOP > 0.90	excellent fidelity
0.75–0.90	acceptable
< 0.75	distribution distorted

NHOP: Formal Definition

Definition (Normalised Histogram Overlap Percentage)

Let $X = \{x_1, \ldots, x_n\}$ and $X' = \{x'_1, \ldots, x'_m\}$ in $\mathbb{R}^k$.
For feature j, form the **joint** range $[\ell_j, u_j]$ over both sets. Divide into B equal-width bins. Let p_b^j, q_b^j be normalised frequencies.

Per-feature:

$$\mathrm{NHOP}_j(X, X') \;=\; \sum_{b=1}^{B} \min(p_b^j,\, q_b^j) \;\in\; [0, 1]$$

Global (mean over features):

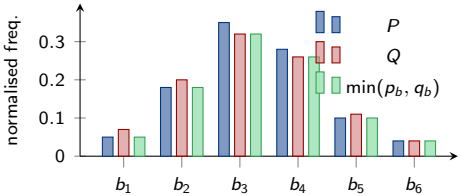
$$\mathrm{NHOP}(X, X') \;=\; \frac{1}{k} \sum_{j=1}^{k} \mathrm{NHOP}_j(X, X')$$

TV Identity (Theorem 4.5):

$$\boxed{\mathrm{NHOP}_j = 1 - \mathrm{TV}(P^j, Q^j)}$$

where $\mathrm{TV}(P, Q) = \frac{1}{2} \sum_b |p_b - q_b|$.

Why joint normalisation?



$\mathrm{NHOP} = 0.05 + 0.18 + 0.32 + 0.26 + 0.10 + 0.04 = 0.95$

Joint range: both P and Q map to the **same** bins, so min is a valid comparison.

NHOP: Mathematical Properties

Analytical Properties

Property	Statement
Boundedness	$\mathrm{NHOP} \in [0,1]$ always
Symmetry	$\mathrm{NHOP}(X, X') = \mathrm{NHOP}(X', X)$
Identity	$X = X' \Rightarrow \mathrm{NHOP} = 1$
Disjoint	$\mathrm{supp}(P) \cap \mathrm{supp}(Q) = \emptyset \Rightarrow 0$
Scale inv.	invariant to affine rescaling of features

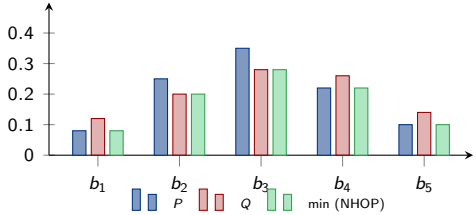
TV Identity — Proof Sketch

Since $\sum_b p_b = \sum_b q_b = 1$:

$$\sum_b \min(p_b, q_b) = 1 - \sum_b \max(p_b - q_b, 0) = 1 - \frac{1}{2} \sum_b |p_b - q_b|$$

The last equality holds because $\sum_b (p_b - q_b) = 0$. Hence $\mathrm{NHOP}_j = 1 - \mathrm{TV}(P^j, Q^j)$. $\square$

NHOP and TV are exact complements



Green area $= \mathrm{NHOP} = 0.88$. Remaining (blue+red not covered) $= \mathrm{TV} = 0.12$. $\mathrm{NHOP} + \mathrm{TV} = 1.0$.

Geometry-Preserving Seed Selection

Problem

Standard oversamplers use every point as a seed — including noisy outliers that distort synthesis.

Composite Score

$$S(C) = \mathrm{NHOP}(C) + \mathrm{AGTP}(C) - w_J\,\mathrm{JSD}(C) - w_Z\,Z(C)$$

Term	Meaning
$\mathrm{NHOP}(C)$	histogram overlap: seed set C vs. full minority
$\mathrm{AGTP} = G_{\mathrm{sim}} + T_{\mathrm{sim}}$	geometry + topology match
$\mathrm{JSD}(C)$	Jensen-Shannon penalty (divergence)
$Z(C) = \sigma/\mu$ of nn-dists	spacing regularity penalty

Impact

Seeds	NHOP
Random	0.766
Geometric	**0.907**

4-Stage Pipeline

1. K-Means on $X_{\min}$
2. PCA projection (optional)
3. Stratified candidate sampling
4. Score & keep best S^*

Code

```
sel = cc.GeometricSeedSelector(
    n_seeds=20,
    n_candidates=100,
    random_state=42)

idx, score = sel.select(X_minority)
```

GVM-CO: Gravity-Biased Von Mises Circular Oversampling

Algorithm

For each seed pair (s_i, s_j) (k-NN):

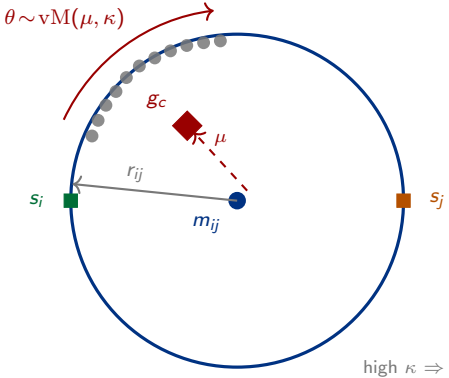
1. Circle: centre m_{ij}, radius $r_{ij} = \frac{1}{2}\|s_i - s_j\|$
2. Angle: $\theta \sim \mathrm{vM}(\mu_{\text{toward } g_c}, \kappa_{ij})$
3. New point: $x' = m_{ij} + r_{ij}(\cos\theta, \sin\theta)$

Gravity concentration

$g_c = \rho_c/(\sigma_c \cdot \bar{d}_c)$,
$s_{ij} = \alpha\tilde{g}_c + (1-\alpha)(1-\bar{d}_{ij}/d_{\max})$

$\kappa_{ij} = \kappa_{\max} \cdot s_{ij}^{\gamma}$ (high gravity $\Rightarrow$ high κ)

```
cc.GVMCO(n_clusters=5,
  kappa_max=4.0, random_state=42)
```



high $\kappa \Rightarrow$ samples concentrate near g_c

LRE-CO: Local Region Estimation Circular Oversampling

Key Idea

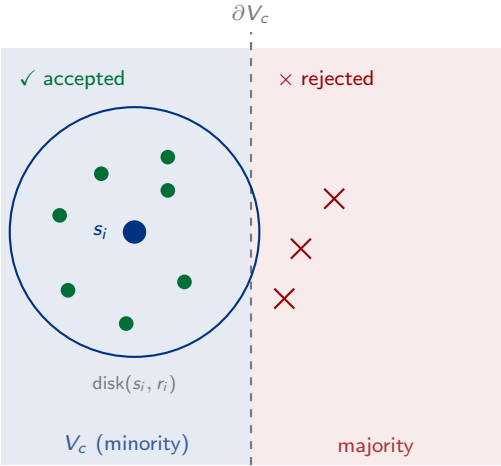
LRE-CO constrains sampling to the **Voronoi cell** of each cluster, preventing spillover into majority territory.

Algorithm

1. K-Means; compute Voronoi cells V_c
2. Certainty check: $P(c \mid s_i) \geq \tau{=}0.80$
3. Sample: $x' \sim \mathrm{Uniform}(\mathrm{disk}(s_i, r_i))$
4. **Reject** if $x' \notin V_c$; repeat
5. Cluster weight: $w_c \propto \tilde{g}_c \cdot n_c$

Guarantee: all synthetic points stay in V_c.

```
cc.LRECO(certainty_threshold=0.80,
  random_state=42)
```



LS-CO: Layered Segmental Circular Oversampling

Structure: L layers $\times$ M segments

Concentric rings (layers) $\times$ angular slices (segments) around each cluster gravity center.

Mathematics

Layer weight:

$$w_\ell \propto \exp\left(-\frac{(\ell-\ell^*)^2}{2\sigma_L^2}\right) \cdot \tilde{g}_c$$

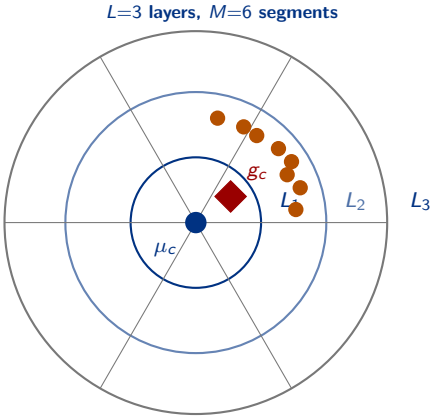
Segment concentration:

$$\kappa_s = \kappa_{\max} \cdot (n_s / n_{\text{layer}})^\gamma$$

Area-uniform radius:

$$r = \sqrt{r_{\text{in}}^2 + u(r_{\text{out}}^2 - r_{\text{in}}^2)}, \; u \sim U[0,1]$$

```
cc.LSCO(n_layers=3, n_segments=6,
  kappa_max=4.0, random_state=42)
```



● sampled, conc. in L_2 near g_c

Algorithm Comparison

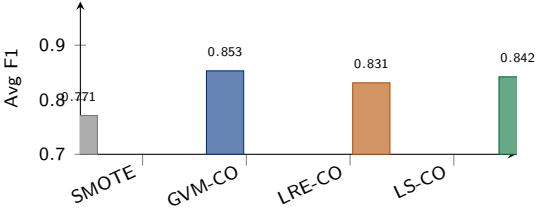
Summary

Method	Region	Bias
GVM-CO	circle (2 seeds)	vM toward g_c
LRE-CO	disk $\cap$ Voronoi	grav. cluster wt.
LS-CO	annular layers	layer wt. + seg. κ

All three

- Drop-in for SMOTE (imblearn API)
- `use_pca=False` native-dim mode
- `random_state` reproducibility
- sklearn CV / pipeline compatible

F1 comparison (14 datasets)



GVM-CO achieves highest avg. F1; all circular methods outperform SMOTE.

Degradation-and-Recovery Benchmark

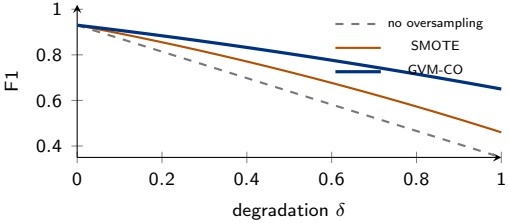
What it tests

1. Remove minority samples in δ steps (0% $\rightarrow$ 100%)
2. Apply oversampler + classifier at each level
3. Record F1/AUC via cross-validation
4. Compute **ARI** $= \int_0^1 \mathrm{score}(\delta)\, d\delta$

Higher ARI = better recovery from imbalance.

```
bench = cc.DegradationBench(
    steps=10, metric="f1",
    cv=5, random_state=42)

# Run on any pipeline
results = bench.run(pipe, X, y)
bench.plot(results)    # recovery curve
ari = cc.DegradationBench\
    .area_recovery_index(results)

# Compare methods
ari_smote = bench.run(smote_pipe, X, y)
ari_gvm   = bench.run(gvm_pipe,   X, y)
```

Recovery curves



Method	ARI
None (baseline)	0.64
SMOTE	0.74
GVM-CO	**0.82**

GitHub Repository and Usage

Links

`github.com/Parsa-Hajian/circover` `pypi.org/project/circover/`

Complete workflow

```python
import circover as cc
from imblearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

# 1. Build pipeline (drop-in for SMOTE)
pipe = Pipeline([
    ("over", cc.GVMCO(random_state=42)),
    ("clf",  RandomForestClassifier()),
])
print(cross_val_score(pipe, X, y,
                      scoring="f1", cv=5).mean())

# 2. Check distribution fidelity
X_res, _ = pipe['over'].fit_resample(X, y)
nhop = cc.NHOP()
print(nhop.score(X[y==1], X_res[len(X):]))

# 3. Run degradation benchmark
bench = cc.DegradationBench(steps=10)
results = bench.run(pipe, X, y)
bench.plot(results)
```

Modules

File	Export
`nhop.py`	`NHOP`
`seed_selection.py`	`GeometricSeedSelector`
`oversampling.py`	`GVMCO, LRECO, LSCO`
`degradation.py`	`DegradationBench`
`_gravity.py`	(internal utils)

Key parameters

Param	Meaning
`n_bins=30`	histogram bins
`n_clusters=5`	K-Means clusters
`kappa_max=4.0`	max vM concentration
`use_pca=True`	PCA mode
`steps=10`	degradation steps

16/16 tests passing `pytest tests/`